

DSA Core Blueprint

Comprehensive Manual: Sequence Rearrangement & Elements Summation

This algorithmic manual provides separate implementations, comprehensive theoretical explanations, and performance metrics for two essential Data Structures and Algorithms (DSA) foundational procedures.

Topic 1: Rearrange Array in Increasing-Decreasing Order

PROBLEM 01

Concept Objective: Given an unsorted array of integers, modify the positions of the values so that the first half of the array elements follows an increasing (ascending) trend line, and the second half follows a decreasing (descending) trend line. This structured setup is known as a *Bitonic Sequence*.

Visual Sequence Mapping

Initial Unsorted Array:

5	4	7	1	9	2
---	---	---	---	---	---

1. Fully Sorted Array (Ascending Order):

1	2	4	5	7	9
---	---	---	---	---	---

2. Final Bitonic Array Output (First 3 Increasing, Last 3 Decreasing):

1	2	4	9	7	5
---	---	---	---	---	---

Python Source Code

```
def rearrange_increasing_decreasing(arr):
    # Find array length and locate exact mid-point index boundary
    n = len(arr)
    mid = n // 2

    # Sort the entire array ascendingly
    arr.sort()

    # Isolate the first half segment
    first_half = arr[:mid]
```

```
# Isolate the second half segment and flip it backwards
second_half = arr[mid:]
second_half.reverse()

# Concatenate segments and output the resulting array
final_result = first_half + second_half
return final_result
```

Step-by-Step Explanation

`n = len(arr) & mid = n // 2 :`

Identifies how many elements exist. Floor integer division (`//`) establishes the threshold where the ascending section stops and the descending section begins.

`arr.sort() :`

Reorders the original collection sequentially from smallest to largest in-place. This step provides an $O(N \log N)$ time bound complexity.

`first_half = arr[:mid] :`

Uses Python array slicing to extract elements from index `0` up to (but excluding) the `mid` index. Since the complete array is sorted, this segment naturally stays in increasing order.

`second_half = arr[mid:] & second_half.reverse() :`

Extracts the rest of the elements starting from the `mid` index to the end. Calling `.reverse()` shifts the elements into a perfectly descending configuration.

`first_half + second_half :`

Glues the two distinct array parts back together into a single list, fulfilling the structural condition.

Topic 2: Calculate Sum of Elements in the Array

PROBLEM 02

Concept Objective: Traverse through an entire linear data array structure systematically, evaluating and accumulating individual numerical items into a single total scalar sum variable value.

Python Source Code

```
def calculate_array_sum(arr):  
    # Initialize an accumulator variable at zero  
    total_accumulator = 0  
  
    # Loop through every element inside the collection  
    for element in arr:  
        # Add current item value directly to accumulator  
        total_accumulator += element  
  
    return total_accumulator
```

Step-by-Step Explanation

total_accumulator = 0 :

Creates a memory location acting as a storage vessel initialized at absolute baseline zero before reading elements.

for element in arr: :

Initiates a clean sequential for-in loop pattern across the array items. The execution engine reads elements from the beginning index position (0) incrementally to the final slot (N-1).

total_accumulator += element :

Applies an addition assignment operator. Each iteration increments the total_accumulator container by the numeric integer value of the active index element.

return total_accumulator :

Once the loop finishes reading all components, it breaks out and yields the verified calculated summation value.

Algorithm Complexity Overview

Time Complexity: $O(N)$. The loop visits each unique item exactly once, making its execution time scale linearly with the input size.

Space Complexity: $O(1)$. It uses an in-place accumulator space without allocating extra linear lists or matrix maps, maximizing storage efficiency.